Let's continue from exactly where you stopped: Acceptance Criteria for Student Registration.

You already have:

- **Epic:** Student Account Management
- **User Story:** As a new student, I want to register so that I can create an account.
- **Acceptance Criteria:** Name is required, email must be valid, password must follow the agreed rules, etc.

Now, what are Tasks and Subtasks? Why do we need them? Who creates them?

1 Imagine you are a manager

You tell a developer:

> "Build the student registration feature."

The developer understands the feature and its rules. But the developer still has work to do:

- Create the registration page.
- Write the code to validate the email.
- Write the code to save the student details.
- Show success and error messages.

These are the actual pieces of work needed to build the feature.

> **A Task** is a piece of work that someone needs to do.

2 What is a Task?

Let's use your existing User Story.

> **USER STORY**
> **Student Registration**
> As a new student, I want to register so that I can create an account.

DEVELOPER TASKS

1. Create the registration form.
2. Write code to validate the name, email and password.
3. Write code to save student details in the database.
4. Display success and error messages.

TESTER TASKS

1. Prepare test cases for registration.
2. Test valid and invalid email addresses.
3. Test missing fields and duplicate email addresses.
4. Record bugs found during testing.

Notice something important: the User Story describes **what the student wants**; Tasks describe **what the team members must do** to deliver it.

3 What is a Subtask?

Sometimes, even a Task contains several smaller pieces of work.

Imagine the developer gets this Task:

> **Task:** Create the registration form.

The developer might divide that work into smaller steps:

Task: Create the registration form

- Subtask 1: Create the name field.
- Subtask 2: Create the email field.
- Subtask 3: Create the password field.
- Subtask 4: Add the Register button.

A Subtask is a smaller piece of work inside a Task.

Think of it like this:

- **Task** = Prepare a meal.
- **Subtasks** = Cut vegetables, cook rice, prepare curry.

You divide the work so that it is easier to understand and complete.

Not every Task needs Subtasks. If a Task is already simple, the team can finish it without breaking it down further.

4 Why do we create Tasks and Subtasks?

Suppose you give your team only this User Story:

> “As a new student, I want to register so that I can create an account.”

The developer and tester know what the feature should do. But they still need to decide and organize the work required to deliver it.

Tasks and Subtasks help them:

- **Know what to do:** Everyone has a clear piece of work.
- **Assign responsibility:** A developer can handle the form, while another handles database work.
- **Track progress:** The team can see which work is completed and which is pending.
- **Avoid forgetting work:** Smaller steps make the work easier to manage.

5 What happens if we don't create Tasks and Subtasks?

Imagine four people are working on registration.

Without tasks, the manager says only, **“Complete registration.”**

- One developer assumes someone else will create the database.
- Another developer starts building the form.
- The tester doesn't know when test cases should be prepared.
- The manager cannot easily tell which parts are completed.

The team may still finish the work, but coordination and progress tracking become harder.

> With Tasks and Subtasks, the team can assign the work, track it, and identify unfinished pieces more easily.

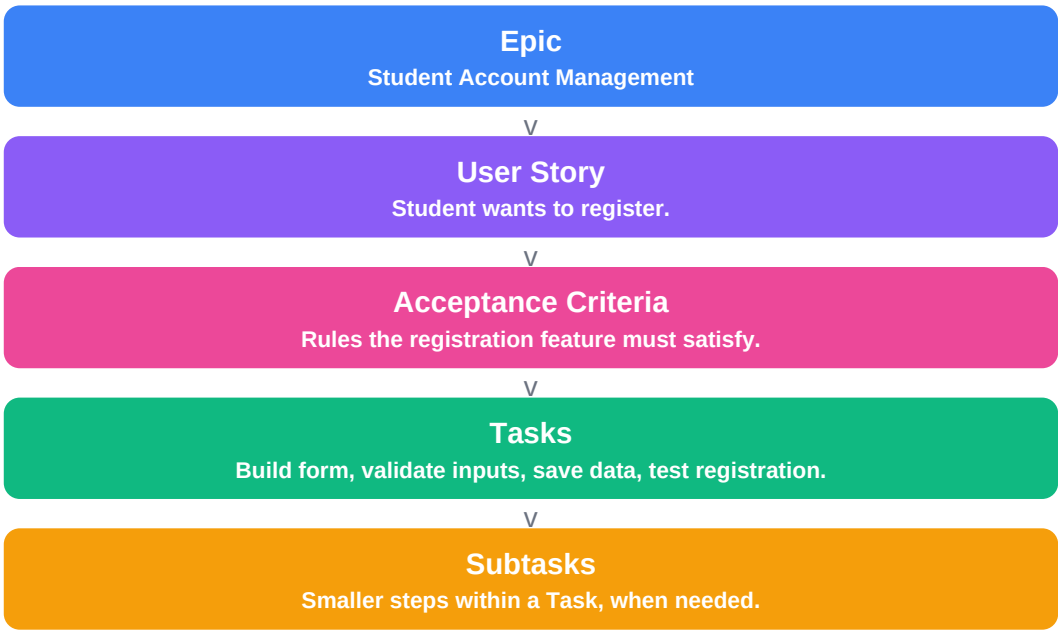
6 Who creates Tasks and Subtasks?

In an Agile team, the exact process can vary by company, but commonly:

Item	Who typically creates or defines it?
Epic	Product Owner, often with input from the team
User Story	Product Owner or Business Analyst, with team input
Acceptance Criteria	Product Owner or Business Analyst, agreed with the team
Developer Tasks	Developers break down the technical work
Tester Tasks	Testers break down the testing work
Subtasks	The person or team responsible for the parent Task usually breaks it down

A Scrum Master may help the team organize its work, but does not necessarily create every task.

7 The complete picture for your registration feature



Remember these five simple questions

- **Epic:** Which big group of work does this belong to?
- **User Story:** What does the user want?
- **Acceptance Criteria:** What rules must the feature satisfy?
- **Task:** What work must someone do?
- **Subtask:** What smaller steps make up that work?

> One final point: Tasks and Subtasks are not mandatory for every team or every User Story. They are used when breaking down the work makes it easier to assign, understand, and track.